Sets Are Rules, Not Buckets: Intensional Programming in C++23 via `dedekind`

The `dedekind` Authors

April 22, 2026

Abstract

We present `dedekind` [1], an open-source C++23 library that embeds formal set-theoretic and categorical structures directly into the type system, available under the Apache-2.0 licence. The central design rule is *intensional first, realize it when you mean it*: a set is represented as a predicate $P : A \to \Omega$ over an ambient species A, not as a heap-allocated container of values. This comprehension view, rooted in Lawvere's Elementary Theory of the Category of Sets (ETCS) [2], has two practical consequences that this paper explores:

- Set operations (intersection, complement, union) compose as predicate combinators at compile time, enabling the LLVM backend to perform *structural pruning*—collapsing logically empty intersections into zero-instruction machine code.
- Numeric realization is deferred to explicit boundaries, cleanly separating abstract algebraic reasoning (e.g. over the naturals $\mathbb{N}$) from hardware-constrained IEEE 754 floating-point [3] semantics.

We motivate the design from the practitioner's perspective, beginning with an analyst data-pipeline notebook, and progressively lifting the underlying abstractions to show how recent C++23 advances (concepts, non-type template parameters, named modules) make this approach viable in a mainstream systems language. We identify *Axiomatic Systems Programming* as a nascent paradigm in which the compiler acts as a formal verification engine, and we leave a precise gap for a cardinality-driven no-op intersection theorem that we conjecture is achievable within the existing type machinery.

Contents

1 Introduction **2**
- 1.1 The Two-Language Problem—From the Analyst's Angle . . . 2
- 1.2 Paper Structure . . . 3

2 Axiomatic Systems Programming **3**
- 2.1 The Substrate: C++23 . . . 3
- 2.2 Structural vs. Nominal Typing: The Juliet Posture . . . 4
- 2.3 The Compiler as Verification and Optimization Engine . . . 5

3 Intensional First, Realize When You Mean It **5**
- 3.1 The Design Rule . . . 5
- 3.2 The Tension: Abstract Naturals vs. IEEE Numerics . . . 5
- 3.3 Dedekind Cuts as the Canonical Example . . . 6

4 Sets Are Rules, Not Buckets **7**
- 4.1 The Comprehension View . . . 7
- 4.2 Set Operations as Predicate Composition . . . 7
- 4.3 The Subobject Classifier Ω . . . 8

5 Compile-Time Structural Pruning **8**
5.1 Structural Pruning in Practice 8
5.2 Algebraic Properties Enable Stronger Pruning 8
5.3 Open Gap: Cardinality-Driven No-Op Intersection 8

6 Related Work **9**

7 Conclusion **10**

1 Introduction

1.1 The Two-Language Problem—From the Analyst's Angle

A data analyst working in Python writes:

```
sales_clean = (
    sales
    .normalize_text(["product_id", "country"], lower=True, strip=True)
    .coerce_numeric(["units", "revenue"], strip_symbols=True)
    .fill_missing(units=0, revenue=0)
    # country is a finite extensional set: ISO 3166-1 alpha-3 codes
    .expect_domain("country", ["DEU", "FRA", "GBR"], on_fail="raise")
    # date is intensional: a point on the real line modelled by ISO 8601
    .expect_iso8601("sale_date")
    # period is a cyclic structure: fiscal quarters repeat annually
    .expect_period("fiscal_quarter", cycle=4)
)

report_plan = monthly_category_report(
    smart_join(sales_clean, products_clean).smart_join(country_meta)
)

summary = report_plan.realize()
```

Listing 1: A declarative analyst pipeline (from the `dedekind` notebook).

Three distinct *kinds* of set structure appear in this single pipeline:

Finite extensional. The call `expect_domain("country", ["DEU", "FRA", "GBR"])` encodes that the `country` column is a member of a known, enumerable set drawn from ISO 3166-1 alpha-3 country codes [4]. The ambient set is finite and fully listed; membership is decidable by equality.

Intensional / infinite. The call `expect_iso8601("sale\_date")` encodes that `sale_date` is a point on a *totally ordered, continuous* ambient structure—the real line $\mathbb{R}$, restricted to the rational subset that ISO 8601 [5] dates inhabit. The ambient set is not enumerated; membership is decided by a parsing predicate. This is the characteristic form of an *intensional* set: defined by a rule rather than a roster.

Cyclic / periodic. The call `expect_period("fiscal\_quarter", cycle=4)` encodes a *cyclic* structure: fiscal quarters are elements of $\mathbb{Z}/4\mathbb{Z}$, a quotient group where quarter 5 wraps back to quarter 1. The ISO 8601 calendar itself exhibits the same periodicity at the week level (52 or 53 ISO weeks per year). Cyclic dimensions are neither purely finite nor purely continuous; their ambient set is a circle, not an interval, and range-based pruning must be replaced by modular arithmetic.

Bootstrapping data quality from guaranteed structural patterns. The three set kinds are not independent. A real-world sales dataset combines all three simultaneously: a country column drawn from a finite ISO 3166-1 roster, a date column inhabiting an intensional time axis, and a period column cycling through fiscal quarters. Their *cross-product* defines a structural skeleton—a lattice of expected cells—against which the actual data can be validated immediately, without any domain-specific business logic.

For example, if a country reports sales data with a known *publication periodicity* (say, Germany publishes monthly, France publishes quarterly), that periodicity is a structural fact expressible as a predicate over the product of the country set and the cyclic period set:

$$\text{Periodicity} = \big\{(c, p) \in \text{ISO3166} \times \mathbb{Z}/n\mathbb{Z} \;\big|\; \text{period\_ok}(c, p)\big\}.$$

A row missing from the expected (c, p) skeleton is a *gap*; a row present outside it is an *anomaly*. Both are detectable as structural violations—set-membership failures—*before* any numeric or business-logic computation is performed.

This turns the three `expect\_*` declarations into a *data quality contract*: the pipeline guarantees, at the boundary of `.realize()`, that every surviving row belongs to the structurally valid region of the joint ambient space. The quality lift reported by the notebook (the delta between the vanilla and smart pivot in the analyst tier) is an empirical measure of how much of the input data falls outside that guaranteed region. Framed this way, data quality improvement is a process of progressively tightening the structural contract—adding more precise predicates—until the residual anomaly rate meets the analyst's tolerance.
The call `.realize()` is an *explicit materialization boundary*: prior to that call, `report_plan` is an intensional description—a recipe without evaluated results. The symmetry between `expect\_*` declarations and `.realize()` is not accidental; it is the central design rule of the `dedekind` library made visible at the Python-facing API tier.

Underneath the Python facade, the library is implemented in C++23. The analyst's pipeline is *not* novel in its high-level form—declarative data-frame APIs are widespread. What is novel is the semantic model that the C++ layer enforces:

1. Sets are *rules* (predicates), not *buckets* (containers).
2. Computation is *intensional until explicitly realized.*
3. The compiler is used as a *verification and optimization engine* for algebraic invariants.

The rest of this paper unpacks these three claims and their consequences for both programmer experience and generated code quality.

1.2 Paper Structure

Section 2 motivates the choice of C++23 as the implementation substrate and introduces the *Axiomatic Systems Programming* paradigm. Section 3 explains the intensional-first design rule and the tension between abstract algebraic numerics and IEEE 754 hardware floating point. Section 4 develops the view of sets as predicates over ambient species and shows how set operations compose symbolically. Section 5 presents compile-time structural pruning, including the open gap for a cardinality-driven no-op intersection. Section 6 surveys related work. Section 7 concludes.

2 Axiomatic Systems Programming

2.1 The Substrate: C++23

The *two-language problem* [6] is the observation that research code is written in an expressive high-level language (Python, R, Julia) while performance-critical execution is delegated to a

separately maintained C or C++ backend. Bridging tools such as `nanobind` [7] close the *binary* gap but not the *semantic* gap: the C++ backend cannot reason about high-level mathematical invariants defined in the Python layer.

Recent advances in C++ change this calculus. Three C++23 features are central:

Concepts allow *structural typing*: a type satisfies a concept if it provides the required operations and relations, regardless of its name. This is the compile-time analogue of Haskell type classes, but with zero-cost dispatch and no runtime dictionary.

Non-type template parameters (NTTP) allow lambdas—including predicate functions—to appear as template arguments. A set-builder predicate can therefore be part of the *type*, not merely a runtime value passed at construction.

Named modules enforce a strict directed-acyclic build graph, preventing circular dependencies between algebraic layers and caching verified structural invariants as Binary Module Interface (BMI) files.

Together, these features enable the compiler to act as a formal verification engine.

2.2 Structural vs. Nominal Typing: The Juliet Posture

Three distinct schools of structural identification appear in the history of programming languages. Understanding their differences clarifies why C++23 concepts represent a genuine advance.

Nominal typing. Traditional object-oriented frameworks require a type to *declare* its membership. A class is a `Ring` because it inherits from a base class named `Ring`, or implements an interface so named. The *name* is the proof of membership. The compiler checks lineage, not behaviour. A type that provides every ring operation but neglects to inherit the right base class is silently excluded; a type that inherits the right class but provides broken operations is silently accepted.

Duck typing. Dynamic languages (Python, Ruby, early JavaScript) use a runtime structural check colloquially known as *duck typing*: "if it walks like a duck and quacks like a duck, it is a duck." Membership is determined at the call site by testing whether the required method exists and can be invoked. This is genuinely structural—it cares about behaviour, not name—but the check happens *at runtime*, too late to drive compile-time optimization, and errors surface only when a missing method is actually called. Python's `expect_domain` constraint in Listing 1 is duck-typed in this sense: the domain set is a runtime `list`, membership is tested by `in`, and a violation raises an exception at the point of evaluation.

Static structural typing—the Juliet Posture. C++23 concepts combine the structural insight of duck typing with the static, compile-time guarantees of nominal typing—without requiring a declaration of intent. We call this the *Juliet Posture*, after the observation that a rose by any other name would smell as sweet [8]: the library cares about the *scent* (structural invariants), not the *lineage* (declared name).

```
// Nominal: recognised by declared lineage
struct Ring { virtual T operator+(T) const = 0; ... };
struct MyInt : Ring { ... };    // must opt in explicitly

// Duck typing: recognised at runtime by the call site
def generic_sum(a, b):          # Python: fails only when called
    return a + b                # no compile-time guarantee
```

```cpp
// Juliet Posture (C++23): recognised statically by behaviour
template <typename T>
concept IsRing = requires(T a, T b) {
    { a + b } -> std::same_as<T>;
    { a * b } -> std::same_as<T>;
    { T::origin() } -> std::same_as<T>; // additive identity
    requires SpeciesTraits<T, std::plus<T>>::is_associative;
};

// int satisfies IsRing without any base class, checked at compile time.
static_assert(IsRing<int>);     // proof obligation discharged at translation
```

Listing 2: The Juliet Posture: structural recognition checked statically, without requiring declared lineage.

The Juliet Posture delivers the best properties of both predecessors: structural sensitivity (behaviour, not name) and static verification (compile time, not runtime). Crucially, because the concept check is a compile-time proof obligation, the compiler can *act on it*: it can specialize code paths, infer additional algebraic properties from the trait registry, and ultimately prune logically empty set expressions to zero machine instructions—none of which is possible with runtime duck typing.

2.3 The Compiler as Verification and Optimization Engine

By encoding algebraic laws as concept requirements, every instantiation of a generic template becomes an implicit proof obligation discharged at compile time. Failed proofs are type errors, not runtime exceptions.

The LLVM backend then exploits this structural knowledge for optimization. When a set is defined as a predicate, and two predicates are logically contradictory, the compiler can determine—at translation time—that the resulting intersection is the empty set, and emit no machine instructions for it. This is what we term *structural pruning*, and it is the subject of Section 5.

3 Intensional First, Realize When You Mean It

3.1 The Design Rule

The central methodology of `dedekind` is captured by a single rule:

> *Intensional first, realize it when you mean it.*

The thrust of this rule is to leverage *lazy evaluation*—both at the type level (compile-time symbolic substitution) and at the value level (deferred numeric approximation)—as a principled defence against *Premature Materialization*: the loss of mathematical structure that occurs when a computation is forced to a concrete representation earlier than necessary.

An *intensional* description specifies a mathematical object by its defining properties. An *extensional* representation enumerates its members or computes its value. The analyst's `report_plan` in Listing 1 is intensional; the returned `summary` data frame is extensional. The `.realize()` call is the auditable crossing of that boundary. This framing is close in spirit to intensional database semantics (for example, Majkić's intensional FOL treatment for peer-to-peer data integration [**majkic2009intensional**]), but our engineering contrast with mainstream SQL is explicit: SQL evaluation is typically defined over already-realized relations, while `dedekind` preserves predicate-level structure in the type system and defers materialization to an explicit realization boundary.

3.2 The Tension: Abstract Naturals vs. IEEE Numerics

The motivating tension is the mismatch between the mathematical naturals $\mathbb{N}$ and the machine integers that implement them.

The ring $\mathbb{Z}$ of integers satisfies:

- Associativity of addition: $(a + b) + c = a + (b + c)$.
- Existence of additive inverse: $\forall a, \exists -a$ such that $a + (-a) = 0$.
- No upper or lower bound.

A 32-bit signed `int` satisfies *none* of these exactly: addition overflows, negation of `INT_MIN` is undefined behavior, and the representable range is finite.

IEEE 754 `double` [3] is even further from any abstract field: addition is non-associative due to rounding, and the special values `NaN` and $\pm\infty$ break the law of excluded middle.

`dedekind` encodes these divergences explicitly in the `:species` trait registry:

```
// Ideal integers: associative addition
template <>
struct SpeciesTraits<int, std::plus<int>> {
    static constexpr bool is_associative = true;
    static constexpr bool is_commutative = true;
};

// IEEE double: addition is NOT bit-exact associative
template <>
struct SpeciesTraits<double, std::plus<double>> {
    static constexpr bool is_associative = false; // rounding
    static constexpr bool is_commutative = true;
};

static_assert( SpeciesTraits<int,    std::plus<int>   >::is_associative);
static_assert(!SpeciesTraits<double, std::plus<double>>::is_associative);
```

Listing 3: The species registry records algebraic properties—and their absence.

Generic algorithms that require associativity will fail to compile for `double` unless an explicit numeric policy (e.g. compensated summation, or an interval-arithmetic wrapper) is supplied. The realization boundary is the point at which the programmer chooses which policy to apply.

3.3 Dedekind Cuts as the Canonical Example

The library's namesake—Dedekind cuts—is the canonical illustration. A real number $r \in \mathbb{R}$ is represented as a pair of intensional sets (L, U) where $L = \{q \in \mathbb{Q} \mid q < r\}$ and $U = \{q \in \mathbb{Q} \mid q \geq r\}$. This is entirely symbolic: no floating-point approximation is involved until the programmer requests a concrete rational bound.

```
// Representing e via the series predicate:
//   e = sum_{n=0}^{inf} 1/n!
// The lower cut L_e = { q in Q | q < e }
using LowerCut_e = Set<Rational, [](Rational q) {
    // q is in L_e iff q < partial_sum(1/n!) for sufficient N
    return series_dominates(q, EulerSeries{});
}>;

// At this point no arithmetic has been performed.
// Comparison e < pi is a compile-time structural check on the cuts.
static_assert(cut_less_than<LowerCut_e, LowerCut_pi>);
```

Listing 4: The Euler number e as an intensional lower cut over $\mathbb{Q}$.

The `static_assert` succeeds because the cuts are formally disjoint subsets of $\mathbb{Q}$ by their predicates, not because any floating-point approximation has been compared.

4 Sets Are Rules, Not Buckets

4.1 The Comprehension View

In `dedekind`, a set is not a heap-allocated container of values—a *bucket*—but a *rule*: a predicate $P : A \to \Omega$ over an *ambient species* A, where Ω is a subobject classifier (classical `bool`, Kleene K3, or another registered logic). The set

$$S = \{x \in A \mid P(x)\}$$

exists as a compile-time type. No elements are stored or enumerated until the programmer explicitly requests materialization.

This is the set-builder, or *comprehension*, representation advocated by Lawvere's ETCS framework [2, 9]: set objects are defined by their membership predicates and universal properties, not by extensional storage.

```
// ambient_set<T>(P) creates the intensional set {x in T | P(x)}.
// No values are stored; the predicate IS the set.
auto even_gt_ten = dedekind::category::ambient_set<int>(
    [](int x) { return x % 2 == 0 && x > 10; }
);

// Membership test: O(1), no heap allocation.
bool b = even_gt_ten.contains(12);   // true
bool c = even_gt_ten.contains(7);    // false
```

Listing 5: A set as a compile-time predicate type.

4.2 Set Operations as Predicate Composition

Because sets are predicates, set operations are predicate combinators:

$$S_1 \cap S_2 \triangleq \{x \in A \mid P_1(x) \wedge P_2(x)\}$$
$$S_1 \cup S_2 \triangleq \{x \in A \mid P_1(x) \vee P_2(x)\}$$
$$\overline{S_1} \triangleq \{x \in A \mid \neg P_1(x)\}$$

These combinators are composed *entirely at compile time*; the resulting predicate is a new type, not a new runtime object. The compiler can therefore reason about it before any program execution occurs.

```
// Two intensional sets over int:
using Primes = Set<int, [](int n) { return is_prime(n); }>;
using Evens  = Set<int, [](int n) { return n % 2 == 0; }>;

// Intersection: the predicate is P_prime AND P_even
using PrimesAndEvens = Intersection<Primes, Evens>;

// The compiler inspects both predicates.
// Primes > 2 are odd; even primes = {2}.
// With sufficient metadata, this reduces at compile time.
```

Listing 6: Set intersection as type-level predicate conjunction.

4.3 The Subobject Classifier Ω

The ambient logic Ω is a pluggable type parameter. Classical Boolean logic ($\Omega = \{\top, \bot\}$) is the default. For floating-point domains, `dedekind` provides a Kleene K3 logic ($\Omega = \{-1, 0, +1\}$) in which the `NaN` singularity maps to the *Unknown* truth value, cleanly restoring the law of excluded middle: $P \vee \neg P = \textit{Unknown}$ is not a paradox in K3.

This pluggability is what allows the same set-builder infrastructure to model both exact integer domains (classical logic) and floating-point approximation domains (Kleene logic) without changing the API surface.

5 Compile-Time Structural Pruning

5.1 Structural Pruning in Practice

The payoff of the comprehension view is that the LLVM backend can *prune* a set expression to a no-op when its predicate is statically unsatisfiable.

```
using Above10 = Set<int, [](int x) { return x > 10; }>;
using Below5  = Set<int, [](int x) { return x < 5;  }>;

// Intersection has predicate: x > 10 AND x < 5
// This is logically unsatisfiable for any integer x.
using Impossible = Intersection<Above10, Below5>;

// The compiler verifies emptiness via static_assert.
// The resulting intersection object compiles to zero machine instructions.
static_assert(set_is_empty<Impossible>);
```

Listing 7: Contradictory predicates collapse to zero instructions.

By hoisting the predicate into the type signature, the C++ concept machinery can call `set_is_empty<Impossible>` at compile time. The generated binary contains no membership-test code for `Impossible`: it is structurally pruned by the type checker before code generation.

Showcase witnesses

Two concrete witnesses from the test suite demonstrate this in the actual `dedekind` API.

Showcase 1 — Diagonal contradiction in $\mathbb{R}^2$. On the diagonal $\{(x, y) \mid x = y\}$, the strip predicate $x > 5 \wedge y < 3$ is contradictory (since $x = y > 5$ forces $y > 5$, violating $y < 3$). The intersection is proven empty by `static_assert`; the exported function compiles to a single `ret i1 false`.

```
inline constexpr auto r2 = cartesian_product(R, R);
using R2Point = typename decltype(r2)::Domain;
inline constexpr auto xy = var<decltype(r2)>;

inline constexpr auto diagonal_in_r2 = Set{xy % r2 | [](const R2Point& p) {
  return p.first.resolve() == p.second.resolve();
}};
inline constexpr auto gt5_lt3_strip = Set{xy % r2 | [](const R2Point& p) {
  return (p.first.resolve() > 5.0) && (p.second.resolve() < 3.0);
}};
inline constexpr auto empty_diagonal_cut = diagonal_in_r2 & gt5_lt3_strip;
using R2Logic = typename decltype(empty_diagonal_cut)::logic_species;

static_assert(empty_diagonal_cut(R2Point{Real<double>{6.0}, Real<double>{6.0}})
              == R2Logic::False);
```

```
extern "C" __attribute__((noinline)) bool impress_empty_diagonal_cut() {
  return empty_diagonal_cut(R2Point{Real<double>{6.0}, Real<double>{6.0}})
         == R2Logic::True;
}
```

Listing 8: Showcase 1 source: diagonal contradiction in $\mathbb{R}^2$.

```
define dso_local noundef zeroext i1 @impress_empty_diagonal_cut() #0 {
entry:
  ret i1 false
}
```

Listing 9: Showcase 1 LLVM IR (`clang++-22 -O2`): constant false return.

Showcase 2 — Lattice/square singleton in $\mathbb{C}$. The natural-number lattice (Gaussian integers, $0 \le \mathrm{Re}, \mathrm{Im} \le 3$) intersected with the square $[0.5, 1.5]^2$ contains exactly $c_3 = 1 + i$. The compiler proves all four membership assertions at translation time and folds the exported function to a single `ret i1 true`.

```
inline constexpr auto c = var<   >;
inline constexpr auto natural_lattice_in_c = lattice<C>.bounded(3);
inline constexpr auto square_c1_c2 = Set{c % C | [](const Complex<double>& z) {
  return (z.real() >= 0.5) && (z.real() <= 1.5)
      && (z.imag() >= 0.5) && (z.imag() <= 1.5);
}};
inline constexpr auto lattice_square_intersection =
    natural_lattice_in_c & square_c1_c2;
using CLogic = typename decltype(lattice_square_intersection)::logic_species;

static_assert(lattice_square_intersection(Complex<double>{1.0, 1.0}) == CLogic::True
static_assert(lattice_square_intersection(Complex<double>{0.0, 1.0}) == CLogic::Fals

extern "C" __attribute__((noinline)) bool impress_lattice_square_singleton() {
  return (lattice_square_intersection(Complex<double>{1.0, 1.0}) == CLogic::True)
      && (lattice_square_intersection(Complex<double>{0.0, 1.0}) == CLogic::False)
      && (lattice_square_intersection(Complex<double>{1.0, 0.0}) == CLogic::False)
      && (lattice_square_intersection(Complex<double>{2.0, 2.0}) == CLogic::False);
}
```

Listing 10: Showcase 2 source: lattice/square singleton in $\mathbb{C}$.

```
define dso_local noundef zeroext i1 @impress_lattice_square_singleton() #0 {
entry:
  ret i1 true
}
```

Listing 11: Showcase 2 LLVM IR (`clang++-22 -O2`): constant true return.

Both showcases are checked into the repository as LLVM IR fixtures and validated in CI (`make ir-fixture-check`).

5.2 Algebraic Properties Enable Stronger Pruning

Beyond interval arithmetic, *algebraic* properties of the predicates provide additional pruning opportunities. The `:species` registry records which operations are associative, commutative, idempotent, and so forth. These traits inform the compiler about structural relationships between sets:

- $S \cap \varnothing = \varnothing$ (intersection with the empty set is empty).

- $S \cap \overline{S} = \varnothing$ (intersection with the complement is empty).
- If $S_1 \subseteq S_2$ then $S_1 \cap S_2 = S_1$ (subset absorption).

Each of these equalities can be discharged as a `static_assert` once the relevant algebraic witnesses are in the trait registry, reducing the intersection to the simpler set at compile time.

5.3 Open Gap: Cardinality-Driven No-Op Intersection

> **Conjecture (Cardinality Pruning).** Let A be a finite ambient set with known compile-time cardinality $|A| = n$, and let $S_1, S_2 \subseteq A$ be intensional sets represented as type-level predicates $P_1, P_2 : A \to \Omega$. If $|S_1| + |S_2| \leq n$—i.e. their combined cardinality cannot exceed the ambient space—then a symbolic constraint solver operating over the ETCS axioms can determine $S_1 \cap S_2 = \varnothing$ at compile time and reduce the intersection type to the empty-set boundary object, generating zero machine instructions.

The analyst pipeline of Section 1 is the concrete motivating instance. The ISO 3166-1 country-code domain `["DEU", "FRA", "GBR"]` has cardinality 3. If the analyst declares two disjoint domain constraints over the same column—say, a "European Union member" predicate and a "non-EU member" predicate applied to the same finite set of codes—the compiler should be able to determine statically that any row satisfying both constraints simultaneously is impossible, and elide the corresponding runtime membership check entirely.

The machinery required exists in parts: the `:boundaries` partition provides the empty-set and ambient-set objects; the `:mereology` partition provides subset witnesses; the `:species` trait registry provides cardinality annotations for finite types. What is missing is a *decision procedure* that combines these three ingredients: given two predicate types and a finite ambient type with known cardinality, determine at compile time whether their intersection is empty.

We leave this as an explicitly open contribution target for future work. The implementation gap is narrow: the required semantic content is already in the library; the missing piece is a `consteval` procedure that traverses the ambient set, evaluates both predicates, and confirms disjointness. For small finite domains (bounded enumerations, flag sets, small integers) this procedure is computationally feasible at compile time and would directly benefit the analyst API tier.

Status (April 2026): This gap is tracked as GitHub issue #348 (`feat(sets): consteval cardinality pruning for disjoint intersection`). The foundation (boundaries, mereology, species registry) is complete; the decision procedure awaits implementation. See also issue #349 for validation via LLVM IR showcase examples.

6 Related Work

Functional and dependently-typed approaches. Agda [10] and Coq provide dependent type systems in which set-membership proofs are first-class terms. The expressiveness is greater than C++23, but the runtime model differs fundamentally: proof terms are erased at runtime, whereas `dedekind`'s structural pruning targets machine-code generation in a systems-programming context. Haskell [11] supports type-class-based structural reasoning, and libraries such as `algebra` encode ring and field hierarchies. C++ templates differ in that they operate on concrete machine types (with hardware-level algebraic imperfections) rather than erased polymorphic dictionaries.

Exact arithmetic. The GNU Multiple Precision library (GMP) and `mpfr` support arbitrary-precision arithmetic at runtime. `iRRAM` [12] and similar libraries implement computable real arithmetic. `dedekind` is complementary: exact reals (Dedekind cuts) are modelled *intensionally as types*, and realization to a specific arithmetic backend is an explicit programmer choice.

Set-builder type systems. Refinement types [13] attach predicates to types, enabling a form of set-builder representation in ML-family languages. Liquid Haskell [14] extends Haskell with SMT-backed refinement predicates. `dedekind` achieves a subset of this functionality in standard C++23 without a custom type checker, using NTTP-embedded lambdas as the predicate carrier.

Data-pipeline DSLs. Apache Spark, Flink, and the Python `pandas`/`polars` ecosystem provide declarative data pipelines. None of these ground their APIs in formal set theory; domain constraints such as `expect_domain` are runtime assertions, not compile-time type invariants. `dedekind`'s Python facade inherits structural guarantees from the C++ layer while preserving the familiar data-frame programming model.

C++ template metaprogramming libraries. Boost.Hana [15] and `mp11` [16] provide compile-time data structure manipulation. `dedekind` builds on these ideas but targets algebraic *semantic* content (associativity, commutativity, cardinality) rather than purely syntactic structure manipulation.

7 Conclusion

We have presented `dedekind`, a C++23 library that treats sets as *rules over ambient species* rather than containers of values. The intensional-first design rule—defer materialization to explicit boundaries—has two direct payoffs: algebraic reasoning is preserved across the abstract/hardware boundary, and the LLVM backend can prune logically empty set expressions to zero machine instructions.

These ideas are motivated from the bottom up, starting with an analyst data pipeline notebook where `.realize()` is the visible realization boundary, and ascending to the formal set-theoretic and categorical foundations that justify the optimization.

Three themes recur throughout:

1. **Axiomatic Systems Programming.** C++23 concepts, named modules, and NTTP-embedded predicates make it possible to use the compiler as a formal verification engine for algebraic laws, without a separate proof assistant.
2. **Intensional first.** The tension between abstract number theory and IEEE 754 hardware semantics is resolved by encoding the divergence in the `:species` trait registry and deferring arithmetic policy choice to the realization boundary.
3. **Sets are rules.** A set defined by a type-level predicate supports compile-time symbolic manipulation that a runtime container cannot.

The open gap of Section 5.3—cardinality-driven no-op intersection for finite ambient sets—represents the next milestone toward making structural pruning a general-purpose optimization pass available to any `dedekind` user.

References

[1] The dedekind Authors. ***dedekind**: Axiomatic Systems Programming in C++23*. `https://github.com/vincentk/dedekind`. Open-source implementation, Apache-2.0 licensed. 2026.

[2] F. William Lawvere. "An elementary theory of the category of sets". In: *Proceedings of the National Academy of Sciences* 52.6 (1964), pp. 1506–1511.

[3] IEEE. *IEEE Standard for Floating-Point Arithmetic*. Provides the formal specification for floating-point formats and operations. IEEE. 2019. DOI: 10.1109/IEEESTD.2019.8766229.

[4] *ISO 3166-1: Codes for the Representation of Names of Countries and Their Subdivisions — Part 1: Country Codes*. Tech. rep. ISO 3166-1:2020. International Organization for Standardization, 2020. URL: https://www.iso.org/standard/72482.html.

[5] *ISO 8601-1: Date and Time — Representations for Information Interchange — Part 1: Basic Rules*. Tech. rep. ISO 8601-1:2019. International Organization for Standardization, 2019. URL: https://www.iso.org/standard/70907.html.

[6] Jeff Bezanson et al. "Julia: A Fresh Approach to Numerical Computing". In: *SIAM Review* 59.1 (2017), pp. 65–98. DOI: 10.1137/141000671.

[7] Wenzel Jakob. *nanobind: tiny and efficient C++/Python bindings*. https://github.com/wjakob/nanobind. 2022.

[8] William Shakespeare. *The Most Excellent and Lamentable Tragedy of Romeo and Juliet*. Act 2, Scene 2. John Danter, 1597.

[9] F. William Lawvere and Robert Rosebrugh. *Sets for Mathematics*. Discusses the construction of the continuum and Dedekind cuts within a categorical framework. Cambridge University Press, 2003.

[10] Ulf Norell. "Towards a practical programming language based on dependent type theory". PhD thesis. Chalmers University of Technology, 2007. URL: http://www.cse.chalmers.se.

[11] The GHC Development Team. *The Glasgow Haskell Compiler User's Guide, Edition 2024*. GHC 2024 Language Edition. 2024. URL: https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/control.html.

[12] Norbert Th. Müller. "The iRRAM: Exact Arithmetic in C++". In: *Computability and Complexity in Analysis (CCA 2000)*. Vol. 2064. Lecture Notes in Computer Science. Springer, 2001, pp. 222–252.

[13] Tim Freeman and Frank Pfenning. "Refinement Types for ML". In: *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. ACM, 1991, pp. 268–277.

[14] Niki Vazou et al. "Refinement Types for Haskell". In: *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming (ICFP)*. ACM, 2014, pp. 269–282.

[15] Louis-Dionne Chagnon. "Boost.Hana: Your Metaprogramming Nightmare". In: *CppCon 2015*. https://github.com/boostorg/hana. 2015.

[16] Peter Dimov. *mp11: A C++11 Metaprogramming Library*. https://github.com/boostorg/mp11. 2017.